

Solving for User Equilibrium

The problem of finding the user equilibrium over a transportation network was introduced in Chapter 1. It involves the assignment of O-D flows to the network links so that travel time on all used paths for any O-D pair equals the minimum travel time between the origin and destination. This problem was formulated as a mathematical program (the equivalent UE minimization) in Chapter 3. Chapter 4 reviewed a general class of algorithms for solving mathematical programs and focused on the convex combinations method. This chapter describes the application of this method to the solution of the user-equilibrium minimization program.

Before this application of the algorithm is explained, however, two of the most common heuristic methods for finding the user-equilibrium flow pattern are outlined in Section 5.1. These approaches had been extensively used before solution algorithms for the UE program were developed and are still widely applied today. Section 5.2 then demonstrates how the convex combinations algorithm can be applied to the equivalent user-equilibrium program. The last section of this chapter concentrates on an algorithm for finding the minimum-travel-time path between two network nodes. The determination of minimum paths is a major component of the algorithmic solution of the UE program (and of the heuristic algorithms mentioned in Section 5.1).

5.1 HEURISTIC EQUILIBRATION TECHNIQUES

The heuristic approaches to the user-equilibrium problem reviewed in this section include *capacity restraint* methods and *incremental assignment* techniques. At the core of these methods lies the *network loading* mechanism.

Network loading is the process of assigning the O–D entries to the network for specific (constant) link travel times. The process follows the route-choice criterion, which underlies any traffic assignment model. As argued in Chapter 1, the UE flow pattern is the result of each motorist using the minimum travel time path between his origin and his destination. Accordingly, the network loading mechanism used in all the algorithms designed to solve the UE problem assigns each O–D flow to the shortest travel time path connecting this O–D pair. As mentioned in Chapter 3 (see discussion following Eqs. [3.28]), this procedure is known as the “all-or-nothing” assignment.

In the all-or-nothing procedure, each O–D pair r – s is examined in turn and the O–D flow, q_{rs} , is assigned to every link that is on the minimum-travel-time path connecting origin r to destination s . All other paths connecting this O–D pair are not assigned any flow. During this process, the link travel times are assumed to be fixed (i.e., not flow dependent) at some value. Accordingly, if there is a performance curve, $t_a(x_a)$, corresponding to each link in the network, any mention of all-or-nothing assignment (or any other network loading mechanism) should be made in conjunction with a specific travel time. For example, the all-or-nothing assignment can be applied to an empty network, that is, using the travel times $t_a = t_a(0)$ for every link, a . The only computational difficulty with the all-or-nothing procedure involves the identification of the minimum-travel-time paths connecting each O–D pair. This, however, is a well-researched problem in graph theory and several algorithms for its solution are readily available. Section 5.3 describes one of the more efficient of these procedures, one that is particularly applicable to transportation networks.

Many of the early urban transportation studies have used the all-or-nothing procedure (based on empty network times) as the traffic assignment procedure. This assignment method does not recognize, of course, the dependence between flows and travel time, thus, in effect, it ignores the equilibrium problem altogether.

Capacity Restraint

In an attempt to capture the equilibrium nature of the traffic assignment problem, transportation planners have devised an iterative scheme known as capacity restraint. This method involves a repetitive all-or-nothing assignment in which the travel times resulting from the previous assignment are used in the current iteration. The algorithm can be summarized as follows:

Step 0: Initialization. Perform all-or-nothing assignment based on $t_a^0 = t_a(0)$, $\forall a$. Obtain a set of link flows $\{x_a^0\}$. Set iteration counter $n := 1$.

Step 1: Update. Set $t_a^n = t_a(x_a^{n-1})$, $\forall a$.

Step 2: Network loading. Assign all trips to the network using all-or-nothing based on travel times $\{t_a^n\}$. This yields a set of link flows $\{x_a^n\}$.

TABLE 5.1 Capacity Restraint Algorithm Applied to the Network in Figure 5.1

Iteration Number	Algorithmic Step	Link		
		1	2	3
0	Initialization	$t_1^0 = 10$ $x_1^0 = 10$	$t_2^0 = 20$ $x_2^0 = 0$	$t_3^0 = 25$ $x_3^0 = 0$
1	Update Loading	$t_1^1 = 947$ $x_1^1 = 0$	$t_2^1 = 20$ $x_2^1 = 10$	$t_3^1 = 25$ $x_3^1 = 0$
2	Update Loading	$t_1^2 = 10$ $x_1^2 = 10$	$t_2^2 = 137$ $x_2^2 = 0$	$t_3^2 = 25$ $x_3^2 = 0$
3	Update Loading	$t_1^3 = 947$ $x_1^3 = 0$	$t_2^3 = 20$ $x_2^3 = 10$	$t_3^3 = 25$ $x_3^3 = 0$
		$\vdots$	$\vdots$	$\vdots$

Step 3: Convergence test. If $\max_a \{|x_a^n - x_a^{n-1}|\} \leq \kappa$,† stop (the current set of link flows is the solution). Otherwise, set $n := n + 1$ and go to step 1.

Table 5.1 demonstrates an application of this procedure to the example network depicted in Figure 5.1, shown on page 114. Starting with travel times corresponding to an empty network, the initial solution is determined and the iterations follow the above-mentioned algorithm. Note that the algorithm does not converge, as the flow “flip-flops” between links 1 and 2, whereas link 3 does not get loaded at all.

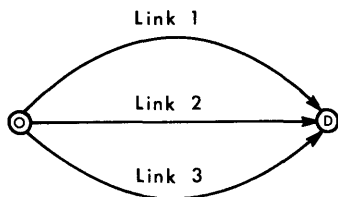
To remedy this situation, the algorithm can be modified as follows. First, instead of using the travel time obtained in the previous iteration for the new loading, a combination of the last two travel times obtained is used. This introduces a “smoothing” effect. Second, the failure to converge is recognized explicitly and the algorithm is terminated after a given number of iterations, N . The equilibrium flow pattern is then taken to be the average flow for each link over the last four iterations (obviously, N should never be less than 4). This form of the algorithm was adopted by the U.S. Federal Highway Administration (FHWA) as part of its transportation planning package. The steps of the modified capacity restraint algorithm (using weights of 0.75 and 0.25 for the averaging process) are as follows:

Step 0: Initialization. Perform an all-or-nothing assignment based on $t_a^0 = t_a(0)$. Obtain $\{x_a^0\}$. Set $n := 1$.

Step 1: Update. Set $\tau_a^n = t_a(x_a^{n-1})$, $\forall a$.

Step 2: Smoothing. Set $t_a^n = 0.75t_a^{n-1} + 0.25\tau_a^n$, $\forall a$.

†This convergence test is based on the maximum change in link flow between successive iterations. Other criteria can also be used.



$$t_1 = 10 \left[1 + 0.15 \left(\frac{x_1}{2} \right)^4 \right] \text{ time units}$$

$$t_2 = 20 \left[1 + 0.15 \left(\frac{x_2}{4} \right)^4 \right] \text{ time units}$$

$$t_3 = 25 \left[1 + 0.15 \left(\frac{x_3}{3} \right)^4 \right] \text{ time units}$$

$$x_1 + x_2 + x_3 = 10 \text{ flow units}$$

Figure 5.1 Network example, with three links and one O-D pair.

Step 3: Network loading. Perform all-or-nothing assignment based on travel times $\{t_a^n\}$. This yields $\{x_a^n\}$.

Step 4: Stopping rule. If $n = N$, go to step 5. Otherwise, set $n := n + 1$ and go to step 1.

Step 5: Averaging. Set $x_a^* = \frac{1}{4} \sum_{i=0}^3 x_a^{n-i} \forall a$ and stop. ($\{x_a^*\}$ are the link flows at equilibrium.)

The smoothing is accomplished by creating a temporary link-travel-time variable, τ_a^n , which is not used as the travel time for the next iteration (see step 1). Instead, it is averaged together with the travel time used in the last iteration, t_a^{n-1} , to obtain the link travel time for the current iteration, t_a^n . This is done in step 2. This algorithm differs from the original capacity restraint algorithm in the addition of the smoothing step and the averaging step. Note that step 4 is called a stopping rule rather than a convergence test, since there is no reason to expect this algorithm to converge to the equilibrium solution, in spite of these changes.

An application of this algorithm to the network example of Figure 5.1 is demonstrated in Table 5.2. Note that it produces a solution that is not an equilibrium flow pattern since, even though all paths are used, t_1^* is substantially different from t_2^* and t_3^* . The table shows three iterations in addition to the initialization, meaning that four minimum-path computations are represented. For large networks, this is the main computational burden, and thus traffic assignment algorithms should be compared in terms of efficiency for a given number of minimum-path computations (i.e., a given number of all-or-nothing assignments).

TABLE 5.2 Modified Restraint Algorithm Applied to the Network in Figure 5.1

Iteration Number	Algorithmic Step	Link		
		1	2	3
0	Initialization	$t_1^0 = 10$	$t_2^0 = 20$	$t_3^0 = 25$
		$x_1^0 = 10$	$x_2^0 = 0$	$x_3^0 = 0$
1	Update	$\tau_1^1 = 947$	$\tau_2^1 = 20$	$\tau_3^1 = 25$
	Smoothing	$t_1^1 = 244$	$t_2^1 = 20$	$t_3^1 = 25$
	Loading	$x_1^1 = 0$	$x_1^1 = 10$	$x_3^1 = 0$
2	Update	$\tau_1^2 = 10$	$\tau_2^2 = 137$	$\tau_3^2 = 25$
	Smoothing	$t_2^2 = 186$	$t_2^2 = 49$	$t_3^2 = 25$
	Loading	$x_2^2 = 0$	$x_2^2 = 0$	$x_3^2 = 10$
3	Update	$\tau_1^3 = 10$	$\tau_2^3 = 20$	$\tau_3^3 = 488$
	Smoothing	$t_1^3 = 142$	$t_2^3 = 42$	$t_3^3 = 141$
	Loading	$x_1^3 = 0$	$x_2^3 = 10$	$x_3^3 = 0$
Average		$x_1^* = 2.5$	$x_2^* = 5.0$	$x_3^* = 2.5$
		$t_1^* = 13.7$	$t_2^* = 27.3$	$t_3^* = 26.8$

Incremental Assignment

Another heuristic method for attaining the user-equilibrium solution assigns a portion of the origin–destination matrix at each iteration. The travel times are then updated and an additional portion of the O–D matrix is loaded onto the network. In this manner, the general shape of the link performance functions can be “traced” with the successive assignments. This procedure, which is known as incremental assignment, is outlined below. (In this description, w_a^n denotes the flow on link a resulting from the assignment of the n th increment of the O–D matrix onto the network.)

Step 0: Preliminaries. Divide each origin–destination entry into N equal portions (i.e. set $q_{rs}^n = q_{rs}/N$). Set $n := 1$ and $x_a^0 = 0, \forall a$.

Step 1: Update. Set $t_a^n = t_a(x_a^{n-1}), \forall a$.

Step 2: Incremental loading. Perform all-or-nothing assignment based on $\{t_a^n\}$, but using only the trip rates q_{rs}^n for each O–D pair. This yields a flow pattern $\{w_a^n\}$.

Step 3: Flow summation. Set $x_a^n = x_a^{n-1} + w_a^n, \forall a$.

Step 4: Stopping rule. If $n = N$, stop (the current set of link flows is the solution); otherwise, set $n := n + 1$ and go to step 1.

In some versions of this algorithm, the incremental all-or-nothing procedure in step 2 is modified and origin–destination pairs are selected in random order,

TABLE 5.3 Incremental Assignment Algorithm Applied to the Network in Figure 5.1

Iteration (Increment)	Algorithmic Step	Link		
		1	2	3
1	Update	$t_1^1 = 10$	$t_2^1 = 20$	$t_3^1 = 25$
	Incremental loading	$w_1^1 = 2.5$	$w_2^1 = 0$	$w_3^1 = 0$
	Summation	$x_1^1 = 2.5$	$x_2^1 = 0$	$x_3^1 = 0$
2	Update	$t_1^2 = 14$	$t_2^2 = 20$	$t_3^2 = 25$
	Incremental loading	$w_1^2 = 2.5$	$w_2^2 = 0$	$w_3^2 = 0$
	Summation	$x_1^2 = 5.0$	$x_2^2 = 0$	$x_3^2 = 0$
3	Update	$t_1^3 = 69$	$t_2^3 = 20$	$t_3^3 = 25$
	Incremental loading	$w_1^3 = 0$	$w_2^3 = 2.5$	$w_3^3 = 0$
	Summation	$x_1^3 = 5.0$	$x_2^3 = 2.5$	$x_3^3 = 0$
4	Update	$t_1^4 = 69$	$t_2^4 = 20.5$	$t_3^4 = 25$
	Incremental loading	$w_1^4 = 0$	$w_2^4 = 2.5$	$w_3^4 = 0$
	Summation	$x_1^4 = 5.0$	$x_2^4 = 5.0$	$x_3^4 = 0$
Travel time at convergence		$t_1^* = 69$	$t_2^* = 27.3$	$t_3^* = 25$

with a flow summation phase (as in step 3) and travel-time update (as in step 4) following each partial assignment (i.e., after each O-D entry is loaded).

Table 5.3 demonstrates the application of this algorithm to the network example of Figure 5.1. The flow pattern resulting from an incremental assignment with four increments is $\mathbf{x} = (5, 5, 0)$. The last row in the table [$t_a^* = t_a(x_a^4)$ for $a = 1, 2, 3$] is provided so that the reader can judge the closeness of the solution to an equilibrium flow pattern. As evident from this table the two used paths (links 1 and 2) do not exhibit equal travel times. Furthermore, these travel times are higher than that of the unused path (link 3).

In conclusion, it is clear that the heuristic methods reviewed in this section either do not converge or produce a set of flows that is not in agreement with the user-equilibrium criterion. It may be reasonable to believe, though, that in general, as the number of increments grows, the incremental assignment algorithm may generate a flow pattern closer to the user-equilibrium condition. A very large number of increments (associated with a considerable computational effort) may be required, however, and even then the method will not always produce the user-equilibrium flow pattern.

5.2 APPLYING THE CONVEX COMBINATIONS METHOD

As the examples of the preceding section demonstrate, the heuristic methods discussed thus far may not converge to the equilibrium solution. It is this failure to converge which motivates the overall approach taken in this book—

formulating the user-equilibrium problem as a mathematical program and solving this program. Chapter 3 dealt with the program formulation and showed that the flow pattern which minimizes the UE equivalent program is, in fact, a user-equilibrium solution. This program includes a convex (nonlinear) objective function and a linear constraint set. Chapter 4 reviewed various feasible direction methods for solving such programs, emphasizing the convex combinations method. This method is especially suitable for solving the equivalent UE program since the direction-finding step can be executed relatively efficiently. This step involves the solution of a linear program, which, in the case of the UE program, has a special structure that simplifies its solution.

Using the notation introduced in Chapter 3, the UE objective function is given by

$$\min z(\mathbf{x}) = \sum_a \int_0^{x_a} t_a(\omega) d\omega \quad [5.1a]$$

subject to

$$\sum_k f_k^{rs} = q_{rs} \quad \forall r, s \quad [5.1b]$$

$$f_k^{rs} \geq 0 \quad \forall k, r, s \quad [5.1c]$$

where x_a is the flow on link a , f_k^{rs} is the flow on path k connecting origin r with destination s , and the incidence relationships $x_a = \sum_{rs} \sum_k f_k^{rs} \delta_{a,k}^{rs}$, $\forall a$, hold.

Applying the convex combinations algorithm to the minimization of the UE program requires, at every iteration, a solution of the linear program (LP)

$$\min z^n(\mathbf{y}) = \nabla z(\mathbf{x}^n) \cdot \mathbf{y}^T = \sum_a \frac{\partial z(\mathbf{x}^n)}{\partial x_a} y_a \quad [5.2]$$

over all feasible values of $\mathbf{y} = (\dots, y_a, \dots)$ (see Eqs. [4.23]). The gradient of $z(\mathbf{x})$ with respect to the link flows at the n th iteration is the link travel-time vector, since $\partial z(\mathbf{x}^n)/\partial x_a = t_a^n$ (see Eq. [3.12a]). The LP objective function at the n th iteration (given by Eq. [5.2]) thus becomes

$$\min z^n(\mathbf{y}) = \sum_a t_a^n y_a \quad [5.3a]$$

subject to

$$\sum_k g_k^{rs} = q_{rs} \quad \forall r, s \quad [5.3b]$$

$$g_k^{rs} \geq 0 \quad \forall k, r, s \quad [5.3c]$$

where $y_a = \sum_{rs} \sum_k g_k^{rs} \delta_{a,k}^{rs}$, $\forall a$, and $t_a^n = t_a(x_a^n)$. In this linear program, y_a is the auxiliary variable representing the flow on link a , while g_k^{rs} is the auxiliary flow variable for path k connecting O-D pair r - s . This program calls for minimizing the total travel time over a network with fixed (not flow-dependent) travel times, t_a^n . The total travel time spent in the network will be minimized by assigning all motorists to the shortest travel-time path connecting their

origin to their destination (see Section 3.5). Such an assignment is performed by the all-or-nothing network loading procedure mentioned earlier. Consequently, the program in Eqs. [5.3] is known as the all-or-nothing program. The core of the all-or-nothing procedure is the determination of the shortest paths between all origins and all destinations. Section 5.3 outlines an efficient method for finding these paths between all the network nodes. At this point, it is sufficient to mention that program [5.3] can be solved efficiently.

To see that the solution of program [5.3] does not involve more than an all-or-nothing assignment, the linearization step of the convex combinations method can be derived by taking the gradient of objective function [5.1a] with respect to path flows (instead of link flow, as in [5.2]). The linearized program then becomes

$$\min z^n(\mathbf{g}) = \nabla_f z[\mathbf{x}(\mathbf{f}^n)] \cdot \mathbf{g}^T = \sum_{rs} \sum_k c_k^{rsn} g_k^{rs} \quad [5.4a]$$

subject to

$$\sum_{rs} \sum_k g_k^{rs} = q_{rs} \quad \forall r, s \quad [5.4b]$$

$$g_k^{rs} \geq 0 \quad \forall k, r, s \quad [5.4c]$$

where c_k^{rsn} is the travel time on path k connecting r and s , at the n th iteration of the algorithm. This program can be decomposed by O-D pair since the path travel times are fixed. The resulting subproblem for pair $r-s$ is given by

$$\min z^n(\mathbf{g}^{rs}) = \sum_k c_k^{rs} g_k^{rs} \quad [5.5a]$$

subject to

$$\sum_k g_k^{rs} = q_{rs} \quad [5.5b]$$

$$g_k^{rs} \geq 0 \quad \forall k \quad [5.5c]$$

This program is obviously minimized by finding the path, m , with the smallest travel time among all paths connecting r and s , and assigning all the flow to it. In other words,

$$g_m^{rs} = q_{rs} \quad \text{if } c_m^{rs} \leq c_k^{rs} \quad \forall k \quad [5.6a]$$

and

$$g_k^{rs} = 0 \quad \text{for all other paths} \quad [5.6b]$$

In case two or more paths are tied for the minimum, any one of them can be chosen for flow assignment.

Once the path flows $\{g_k^{rsn}\}$ are found, the link flows can be calculated by using the incidence relationships, that is,

$$y_a^n = \sum_{rs} \sum_k g_k^{rsn} \delta_{a,k}^{rs} \quad \forall a$$

This solution defines the descent direction $\mathbf{d}^n = \mathbf{y}^n - \mathbf{x}^n$.

The remaining algorithmic steps of the convex combinations method are similar to those outlined in Section 4.3. The initial solution can be usually determined by applying an all-or-nothing network loading procedure to an empty network, in a way similar to the initialization of the capacity restraint methods discussed in Section 5.1.

The line search for the optimal move size can be performed with any of the interval reduction methods, but the bisection method (Bolzano search) may be particularly applicable. The reason is that the derivative of the objective function $z[\mathbf{x}^n + \alpha(\mathbf{y}^n - \mathbf{x}^n)]$ with respect to α is given by

$$\frac{\partial}{\partial \alpha} z[\mathbf{x}^n + \alpha(\mathbf{y}^n - \mathbf{x}^n)] = \sum_a (y_a^n - x_a^n) t_a[x_a^n + \alpha(y_a^n - x_a^n)] \quad [5.7]$$

which can be easily calculated for any value of α .

The stopping criterion for solving the UE program could be based on the values of the objective function. As mentioned in Section 3.1, however, this function is merely a mathematical construct that lacks behavioral or economic meaning. Consequently, the convergence criterion should be based on the relevant figures of merit, which in this case consist of the flows and the travel times. A possible measure of the closeness of a particular solution to equilibrium is the similarity of successive O-D travel times. Letting u_{rs}^n denote the minimum path travel time between O-D pair $r-s$ at the n th iteration, the algorithm can terminate if, for example,

$$\sum_{rs} \frac{|u_{rs}^n - u_{rs}^{n-1}|}{u_{rs}^n} \leq \kappa \quad [5.8a]$$

Alternatively, a criterion that is based on the change in flows can be used. For example, the algorithm can terminate if

$$\frac{\sqrt{\sum_a (x_a^{n+1} - x_a^n)^2}}{\sum_a x_a^n} \leq \kappa' \quad [5.8b]$$

Other convergence criteria can be used as well. The algorithm itself, when applied to the solution of the UE problem, can be summarized as follows:

Step 0: Initialization. Perform all-or-nothing assignment based on $t_a = t_a(0)$, $\forall a$. This yields $\{x_a^1\}$. Set counter $n := 1$.

Step 1: Update. Set $t_a^n = t_a(x_a^n)$, $\forall a$.

Step 2: Direction finding. Perform all-or-nothing assignment based on $\{t_a^n\}$. This yields a set of (auxiliary) flows $\{y_a^n\}$.

Step 3: Line search. Find α_n that solves

$$\min_{0 \leq \alpha \leq 1} \sum_a \int_0^{x_a^n + \alpha(y_a^n - x_a^n)} t_a(\omega) d\omega$$

Step 4: Move. Set $x_a^{n+1} = x_a^n + \alpha_n(y_a^n - x_a^n)$, $\forall a$.

Step 5: Convergence test. If a convergence criterion is met, stop (the current solution, $\{x_a^{n+1}\}$, is the set of equilibrium link flows); otherwise, set $n := n + 1$ and go to step 1.

Note the similarity between this algorithm and the capacity restraint method reviewed in Section 5.1. In fact, if the move size, α_n , is fixed at $\alpha_n = 1$ for all n , the resulting algorithm is identical to the capacity restraint method. This observation is important because it means that existing computer programs that use the capacity restraint method can be easily modified to give a convergent algorithmic solution to the UE problem (by inserting step 3).

To demonstrate the convergence of this algorithm to the equilibrium solution, it is applied to the three-link network example depicted in Figure 5.1. The iterations of this algorithm are shown in Table 5.4. The results in this table should be compared to the convergence patterns of the heuristic methods shown in Tables 5.1 to 5.3. From Table 5.4 it is evident that after five iterations (six minimum-path calculations, including the initialization process) the flows are close to equilibrium; the travel times on all three routes are very similar.

A comparison of this algorithm with the previous (heuristic) methods, in

TABLE 5.4 Convex Combinations Algorithm Applied to the Network in Figure 5.1

Iteration Number	Algorithmic Step	Link			Objective Function	Step Size
		1	2	3		
0	Initialization	$t_1^0 = 10.0$ $x_1^1 = 10.00$	$t_2^0 = 20.0$ $x_2^1 = 0.00$	$t_3^0 = 25.0$ $x_3^1 = 0.00$		
1	Update Direction Move	$t_1^1 = 947.0$ $y_1^1 = 0$ $x_1^2 = 4.04$	$t_2^1 = 20.0$ $y_2^1 = 10$ $x_2^2 = 5.96$	$t_3^1 = 25.0$ $y_3^1 = 0$ $x_3^2 = 0.00$	$z(\mathbf{x}) = 1975.00$	$\alpha_1 = 0.596$
2	Update Direction Move	$t_1^2 = 35.0$ $y_1^2 = 10$ $x_1^3 = 3.39$	$t_2^2 = 35.0$ $y_2^2 = 0$ $x_2^3 = 5.00$	$t_3^2 = 25.0$ $y_3^2 = 0$ $x_3^3 = 1.61$	$z(\mathbf{x}) = 197.00$	$\alpha_1 = 0.161$
3	Update Direction Move	$t_1^3 = 22.3$ $y_1^3 = 10$ $x_1^4 = 3.62$	$t_2^3 = 27.3$ $y_2^3 = 0$ $x_2^4 = 4.83$	$t_3^3 = 35.3$ $y_3^3 = 0$ $x_3^4 = 1.55$	$z(\mathbf{x}) = 189.98$	$\alpha = 0.035$
4	Update Direction Move	$t_1^4 = 26.1$ $y_1^4 = 0$ $x_1^5 = 3.54$	$t_2^4 = 26.3$ $y_2^4 = 0$ $x_2^5 = 4.73$	$t_3^4 = 25.3$ $y_3^4 = 10$ $x_3^5 = 1.72$	$z(\mathbf{x}) = 189.44$	$\alpha = 0.020$
5	Update Direction Move	$t_1^5 = 24.8$ $y_1^5 = 10$ $x_1^6 = 3.59$	$t_2^5 = 25.8$ $y_2^5 = 0$ $x_2^6 = 4.70$	$t_3^5 = 25.4$ $y_3^5 = 0$ $x_3^6 = 1.71$	$z(\mathbf{x}) = 189.33$	$\alpha = 0.007$
	Update	$t_1^6 = 25.6$	$t_2^6 = 25.7$	$t_3^6 = 25.4$	$z(\mathbf{x}) = 189.33$	

terms of computational efficiency, can be made on the basis of the flow pattern at the end of the third iteration (this is equivalent to four all-or-nothing assignments in terms of computational effort). At this point, the travel times generated by the convex combinations algorithm on the three paths are $t^4 = (26.1, 26.3, 25.3)$. This situation is much closer to equilibrium than the path travel times generated by the modified capacity restraint method, $t^* = (13.7, 27.3, 26.8)$, or the ones produced by the incremental assignment method, $t^* = (69.0, 27.3, 25.0)$.

As Table 5.4 shows, flow is taken away from congested paths and assigned to less congested paths, at each iteration of the convex combinations method. This process equalizes the travel times among all paths and brings the system toward equilibrium. The reduction in the objective function value illustrated in Table 5.4 follows the pattern demonstrated in Figure 4.9. Again, the marginal contribution of each successive iteration to the reduction in the objective function value is decreasing.

In solving the UE program over a large network, each iteration involves a significant computational cost, due primarily to the effort required to calculate the shortest paths in the direction-finding step. It is important, then, that a good answer is achieved after a relatively small number of iterations.

In practice, this is not a major problem because of two reasons. First, the convergence pattern of the convex combinations algorithm is such that the first few iterations are the most "cost effective." In other words, the flow pattern after only a few iterations is not very far from equilibrium. Second, the convergence criteria used in practice are not very stringent and thus convergence can be achieved after only a small number of iterations. This is because the accuracy of the input data does not warrant the effort needed to obtain an extremely accurate equilibrium flow pattern.

The number of iterations required for convergence is significantly affected by the congestion level on the network. In relatively uncongested networks, a single iteration may suffice since the link flows may be in the range where the performance functions are almost flat. This means that the updated travel times are very close to the initial ones, generating a set of link flows that is quite similar to the initial solution. As congestion builds up, more iterations are required to equilibrate the network. This effect is demonstrated in Figure 5.2, which depicts the convergence pattern of the convex combinations method for a medium-sized network, for three congestion levels. The convergence measure depicted in this figure is based on the objective function values. The three curves in the figure correspond to low, medium, and high levels of congestion over the network. As the figure shows, the congestion effect on the convergence rate can be quite pronounced.

In actual applications, only four to six iterations are usually sufficient to find the equilibrium flow pattern over large urban networks. This number reflects common practice in terms of trade-offs among analytical accuracy, data limitations, and budget, given typical congestion levels.

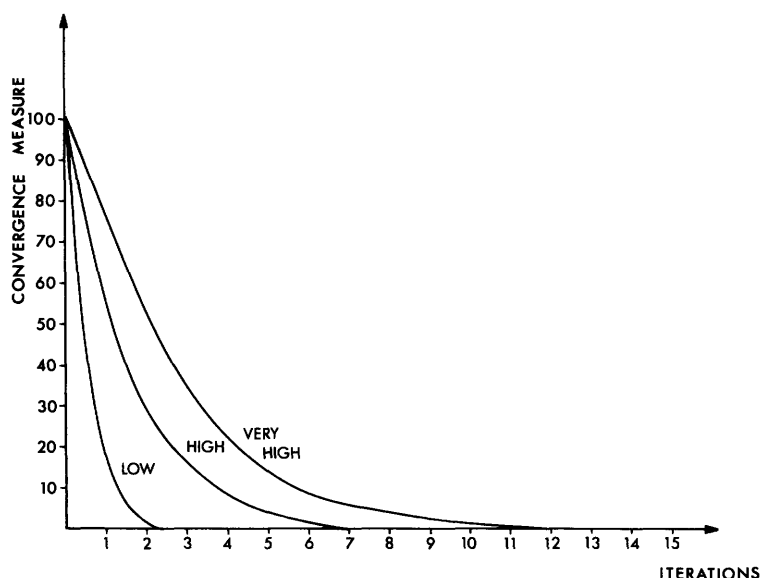


Figure 5.2 Convergence rate of the convex combinations algorithm for various congestion levels. Congestion is measured in terms of $\sum_a x_a t_a(x_a) / \sum_a x_a t_a(0)$, where x_a and t_a are the equilibrium flow and travel times. Convergence is measured by the value of the objective function, normalized between the initial iteration and the 30th one. At iteration n this measure is $\{1 - [z(x^0) - z(x^n)] / [z(x^0) - z(x^{30})]\}100$.

5.3 SHORTEST PATHS OVER A NETWORK

As demonstrated in this chapter, both the optimization and heuristic approaches to the solution of the UE problem require an iterative all-or-nothing assignment. This assignment includes loading the trips between each origin-destination pair on to the shortest travel time path connecting this pair. The problem is thus that of finding the minimum-travel-time paths connecting each O-D pair for a given set of link travel times. The solution algorithm has to be very efficient since, in the course of finding the user equilibrium, the minimum-path problem has to be solved over and over again for each O-D pair at different flow levels.

The notation used up to this point will be slightly modified for the purposes of the discussion in this section. Computer memory, when storing a network, does not keep separate lists of nodes and links, but rather a list of nodes only. Links are identified by their end nodes. If i and j are nodes (i.e., $i, j \in \mathcal{N}$), the link pointing from i to j is indexed by ij (i.e., $ij \in \mathcal{A}$). Accordingly, x_{ij} and t_{ij} denote the flow and travel time, respectively, on link ij .

Shortest-Path Algorithm

The algorithm presented here is known in the operations research literature as the label-correcting method. It finds the shortest path from a given origin (root) node to all other nodes in the network. Accordingly, it has to be

used for each origin in turn for a complete all-or-nothing assignment to be performed. The explanation that follows pertains to the calculation of such a minimum-path *tree*, that is, the shortest path from one root node to all other nodes.

The algorithm essentially scans the network nodes in an iterative manner. At each iteration the algorithm tries to find a path from the root to the node being scanned that is better (shorter) than the current path. The algorithm terminates when no better path can be found from the root to any of the other nodes in the network.

Assume that the network is stored in the computer as a list of links identified by their end nodes. A travel time (or length) t_{ij} is associated with each link ij . In addition, two pieces of information are stored for every node i in the network: 1) the (current) *label* of this node, l_i , and 2) its (current) *predecessor* node, p_i . The label of node i is the distance from the root node to node i along the (current) shortest path. The predecessor of node i is the node just preceding node i along the (current) shortest path. A list of the predecessor nodes (the p_i 's) is continuously updated so that the minimum paths can be traced once the algorithm is terminated. (These paths are traced backward from every node to the root.) The algorithm requires an examination of all the network nodes at least once. To help manage and keep track of the nodes, the algorithm uses an additional list called the sequence list. This list includes all the nodes that have yet to be examined as well as the nodes requiring further examination.

The algorithm is initialized by setting all labels (which are arranged in a label list) to infinity (or, in practice, to a very large number), setting all predecessor nodes (in the predecessor list) to zero, and placing the origin node, r , on the sequence list with label $l_r = 0$.

Each iteration starts with the selection of a node (say, i) from the sequence list for examination. (At the first iteration, the root node is the only one on the sequence list and therefore is examined first.) All nodes, j , that can be reached from i by traversing only a single link are tested in the examination process. If the minimum path to j through i is shorter than the previous path to j , then l_j is updated. In other words, if

$$l_i + t_{ij} < l_j \quad [5.9]$$

then the current shortest path from the root node to j can be improved by going through node i . To reflect this change, the label list is updated by setting $l_j := l_i + t_{ij}$, the predecessor list is updated by setting $p_j := i$, and the sequence list is updated by adding j to it (since this change may affect nodes that can be reached from j). Once all the nodes j (that can be reached from i) are tested, the examination of node i is complete and it is deleted from the sequence list. The algorithm terminates when the sequence list is empty. At this point, the shortest path from the root to any other node can be found by tracing the predecessor list back to the root node.

Note that the algorithm “fans out” from the root node. It first places all

the nodes that can be reached from the origin by traversing only a single link on the sequence list, and eliminates the origin itself from the list. The algorithm then keeps adding nodes to the list if the label test is met and deleting nodes already examined. Note that a node eliminated from the sequence list can reappear on it in a later stage.

Example

Consider the simple network depicted in Figure 5.3. This network includes one O-D pair and four links. (The number shown next to each link indicates the travel time associated with it.) The label-correcting algorithm is used below to find the shortest path from node 1 to node 4.

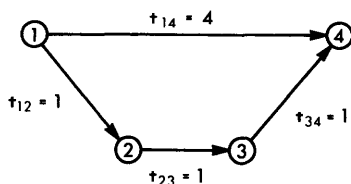


Figure 5.3 Network example for the minimum-path algorithm.

First, all labels are initialized to ∞ and all predecessors to zero. Next, the label of node 1 is changed to zero and it is placed on the sequence list. The sequence list is then scanned and node 1 is chosen for examination (at this point there are no other choices). Two nodes can be reached from node 1 (2 or 4); assume that 4 is considered first. Now, since $l_1 + t_{14} = 4 < l_4 = \infty$, the label of node 4 is changed to 4 and this node is placed on the sequence list. Node 1 is not yet erased from the list, since link $1 \rightarrow 2$ must be considered next. This is then followed by considering links $2 \rightarrow 3$ and $3 \rightarrow 4$. The contents of the label list, the predecessor list, and the sequence list for these iterations are given in Table 5.5. At the fourth iteration, the label of node 4 is changed from 4 to 3. The fifth iteration only verifies that no link emanates from node 4. This node is then removed from the sequence list and the algorithm terminates (since the sequence list is empty). The minimum path can now be traced backward from node 4 by using the predecessor list.

TABLE 5.5 Contents of the Label, Prediction, and Sequence Lists

Iteration	Link Tested	Label List				Predecessor List				Sequence List
		Node 1	Node 2	Node 3	Node 4	Node 1	Node 2	Node 3	Node 4	
Initialization	—	0	∞	∞	∞	0	0	0	0	1
1	$1 \rightarrow 4$	0	∞	∞	4	0	0	0	1	1, 4
2	$1 \rightarrow 2$	0	1	∞	4	0	1	0	1	2, 4
3	$2 \rightarrow 3$	0	1	2	4	0	1	2	1	3, 4
4	$3 \rightarrow 4$	0	1	2	3	0	1	2	3	4
5	—	0	1	2	3	0	1	2	3	

Note on Computer Implementation†

The label-correcting algorithm discussed here can efficiently determine the minimum-travel-time path from an origin node to all other nodes in a network. The algorithm can be applied to very large networks, but care must be taken in coding the algorithm. This section discusses some computer implementation issues and gives some guidelines for efficient coding of the label-correcting method.

The first issue of concern is the method of storing the network. In describing the algorithm, it was assumed that a list of the network links is available. Consider, for example, the network depicted in Figure 5.4 on page 126. This network can be described by the list of its links identified by their end nodes. For example:

"From" Node	"To" Node
1	2
2	3
3	6
1	4
4	5
5	6
1	5
5	2
2	6
2	5

In addition, each link is associated with some travel time, as shown in the figure. At each iteration of the algorithm, all links emanating from a particular node have to be tested. Thus, to avoid repeated searches of the link list, it is advantageous to sort the links so that all links emanating from the same node are stored adjacently; that is,

"From" Node	"To" Node
1	2
1	4
1	5
2	3
2	6
2	5
3	6
4	5
5	2
5	6

†This section can be skipped without loss of continuity.

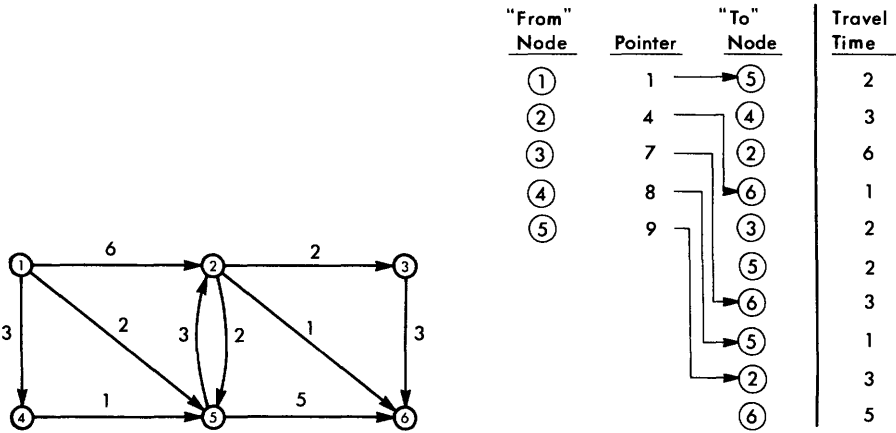


Figure 5.4 Network example demonstrating a forward star network presentation. (The numbers on the links show the links' travel times.)

where "from" nodes are arranged in ascending order. To save space in computer memory, this list can be stored in a "forward star" form, where each arc is represented only by its ending node. A pointer is kept for each node, indicating the position, in the link list, of links beginning with this node. The right-hand side of Figure 5.4 demonstrates this pointer representation. For the example under consideration, this pointer can be thought of as an array, **B**, of length 5 with the following entries:

$$\begin{aligned} B(1) &= 1 \\ B(2) &= 4 \\ B(3) &= 7 \\ B(4) &= 8 \\ B(5) &= 9 \end{aligned}$$

Instead of keeping two link-length arrays,[†] the network can thus be stored with one link-length and one node-length array. Using this forward star form, all the links emanating from each node can be processed easily.

The management of the sequence list is another area in which significant computational gains can be made. First, note that *all* the nodes that represent links emanating from a particular node under examination should be tested consecutively. This keeps the sequence list small and avoids unnecessary searches for the location of a particular link.

To avoid duplicating the computation, nodes should be added to the sequence list only if they are not already on it. As it turns out, it is advanta-

[†]A link-length array includes as many components as the number of links in the network. Similarly, a node-length array includes as many entries as there are nodes in the network.

geous to process the sequence list (i.e., to choose candidate nodes for examination) in a particular order. In general, the sequence list is processed from the top down, and nodes are added to the list by placing them at the bottom. (In other words, the sequence is treated as a queue.) If, however, a node to be added has already been on the list (and examined and removed from it), it should be placed on the top (meaning that it would be examined next). This strategy of managing the sequence list has been shown to be most effective for computing shortest paths over transportation networks.

Figure 5.5 depicts a flowchart of the label-correcting algorithm which utilizes the forward star form and uses the aforementioned strategy for managing the sequence list. Note that, in the computer, this list is not managed by physically moving all nodes in order to place a node at the top of the list. Instead, the sequence list is a node-length array, say s , containing the following information regarding the status of each node in the sequence list:

$$s(i) = \begin{cases} -1 & \text{if node } i \text{ was previously on the list but} \\ & \text{is no longer on it} \\ 0 & \text{if node } i \text{ has never been on the list} \\ +j & \text{if node } i \text{ is on the list and } j \text{ is the next} \\ & \text{node on the list} \\ +\infty & \text{if node } i \text{ is on the list and it is the last} \\ & \text{node on the list} \end{cases}$$

In addition, the top and bottom of the list are identified by special pointers. Placing a node, j , at the top of the list means only that $s(j) := m$ if m was the previous first node on the list, and the top pointer is changed to point at j . Placing a node, j , at the bottom of the list is done by setting $s(j) := \infty$, $s(n) := j$ if n was the previous last node on the list, and the bottom pointer is changed to point at j .

This concludes the description of the shortest path algorithm. The remainder of this section discusses some related numerical issues regarding the solution of user-equilibrium problems.

Transportation networks are characterized by a number of links/number of nodes ratio of approximately 4. For such networks, the computational effort associated with the identification of each minimum-path tree grows linearly (on the average) with the size of the network analyzed. In other words, the computer CPU time† needed is proportional to the number of nodes (and links) in the network. This observation can be used to assess the computational effort associated with the solution of the UE program (using the algorithm outlined in Section 5.2).

Finding the minimum paths is the most computation-intensive component of each iteration of most UE solution procedures (and, in particular, the convex combinations algorithm). The other components (loading the mini-

†CPU (central processing unit) time is a measure of the effort required by the computer to execute a program.

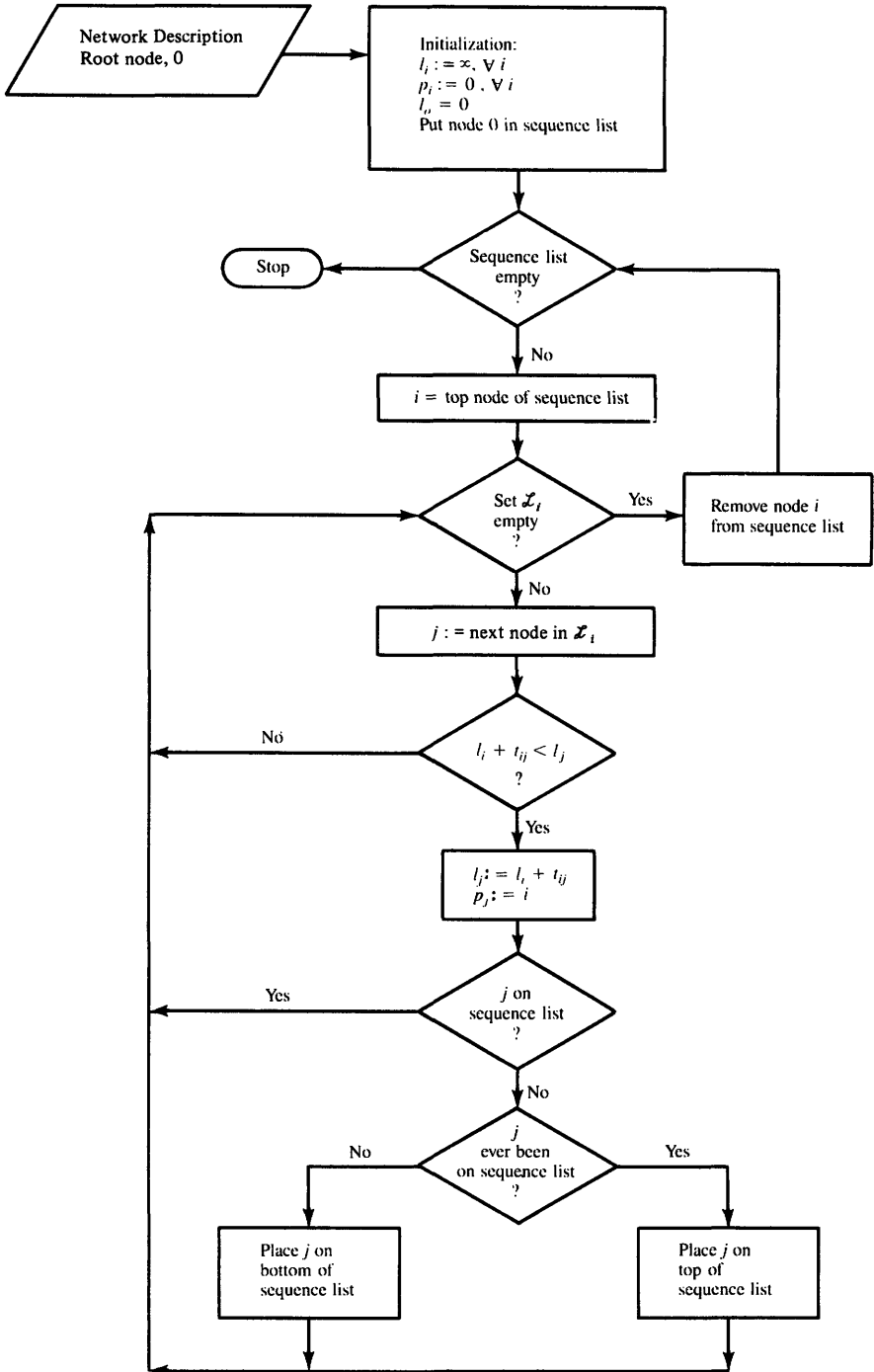


Figure 5.5 Flowchart of the label-correcting shortest-path algorithm. (The set $\mathcal{L}_i$ includes all nodes that can be reached from i by traversing a single link.)

num paths, the line search, the updates, and the convergence checks) do not require more than a few percentages of the total CPU time. Consequently, the computational effort associated with this application of the convex combinations algorithm is proportional to the product of the number of iterations, the number of origins[†] (which determines the number of minimum-path trees to be calculated at each iteration) and the number of nodes (which determines the effort needed to calculate each tree). In other words,

$$\left(\begin{array}{c} \text{computational} \\ \text{costs} \end{array} \right) = K \left(\begin{array}{c} \text{number of} \\ \text{iterations} \end{array} \right) \left(\begin{array}{c} \text{number of} \\ \text{origins} \end{array} \right) \left(\begin{array}{c} \text{number of} \\ \text{nodes} \end{array} \right) \quad [5.10]$$

where K is a constant of proportionality that is computer specific.

5.4 SUMMARY

The focus of this chapter is on finding the user-equilibrium flow pattern over a transportation network. The first section reviews the two most widely used heuristic techniques for solving this network equilibrium problem. These include capacity restraint methods, which are not guaranteed to converge, and incremental assignment methods, which may “converge” to a nonequilibrium solution. The inadequate performance of these heuristics has motivated the development of the equivalent minimization approach used in this text.

The convex combinations algorithm described in Section 4.3 can be easily applied to the solution of the UE equivalent minimization program. The solution of the linear program associated with each step of this algorithm requires merely an all-or-nothing network loading. Such a loading involves the assignment of all flow between each O-D pair to the minimum-travel-time path connecting this O-D pair. The problem of finding the set of all relevant minimum paths is a well-known problem in operations research for which many efficient algorithms are available.

One of the most efficient algorithms for finding the minimum path from a single node to all other network nodes is the label-correcting algorithm described in Section 5.3. This algorithm is extremely efficient and its execution can be expedited even further by careful coding and list-processing procedures.

The use of the convex combinations method in conjunction with the label-correcting (or any other minimum-path) algorithm for the direction-finding step provides an easy and efficient approach to the minimization of the equivalent UE program. The convergence of the convex combinations method is asymptotic in nature; the marginal contribution of each additional iteration

[†]If the number of destinations is smaller than the number of origins, the all-or-nothing assignment can be carried out by rooting the minimum-path trees at the destinations. Each tree would then give the minimum path from each of the network nodes to the destination root. The total computational costs associated with solving the UE program are, then, proportional to the product of the number of iterations, the number of nodes, and the minimum of the number of origins and number of destinations.

to the reduction in the value of the objective function is decreasing. The number of iterations required for convergence is primarily a function of the congestion over the network. The computational effort needed for each iteration is proportional to the number of origins (or number of destinations, if it is smaller) and the size of the network.

5.5 ANNOTATED REFERENCES

The two heuristic method presented in Section 5.1 are embedded in many of the early transportation planning computer packages. The U.S. Federal Highway Administration (FHWA) includes a modified capacity restraint method in its Urban Transportation Planning Package (Federal Highway Administration, 1977). The incremental assignment method was used in The DODO-TRANS package developed at M.I.T. by Manheim and Ruiter (1970). Both types of procedures were criticized by a number of researchers on the grounds mentioned in this chapter. For example, Sheffi and Daganzo (1978) showed the divergence property of the capacity restraint method for a contrived network, while Ferland et al. (1975) demonstrated the problems associated with the use of incremental assignment. The example used in Section 5.1 to demonstrate both the problems associated with the use of heuristic methods and the convergence of the convex combinations method follows the exposition by Eash et al. (1979). Currently, the network assignment package UROAD, which is part of the UMTA Transportation Planning System (UTPS) supported by the U.S. Urban Mass Transportation Administration, includes the convex combinations algorithm.

The application of Frank and Wolfe's convex combinations method to the solution of transportation network equilibrium was first suggested by Bruynooghe et al. (1968) and applied by Murchland (1969). Shortly thereafter it was used by LeBlanc et al. (1975), who coded and tested the algorithm for a small city. At the same time, Nguyen (1974b) suggested the use of the convex simplex method for solving the UE equivalent minimization program. Nguyen (1974a) also suggested the use of the reduced gradient method and a modified reduced gradient method for this purpose. In some side-by-side comparative experiments, Florian and Nguyen (1976) found that even though the convex simplex method converges somewhat faster than the convex combinations method, it requires more computer memory. Consequently, the overall computational effort required by both methods is similar. The latter reference also includes an interesting validation study, where the equilibrium flows were found to be in satisfactory agreement with ground counts in the city of Winnipeg, Canada. Bovy and Jansen (1981) also demonstrate a high level of agreement between ground counts taken in the city of Eindhoven, the Netherlands, and the flows resulting from a user-equilibrium assignment. Figure 5.2 is taken from Mimis (1984), who used a "hub and spokes" simulated network to study some numerical issues in conjunction with equilibrium assignment algorithms.

The minimum-path algorithm presented in Section 5.3 is the label-correcting algorithm suggested by Moore (1957), with the modification for handling the sequence list suggested by Pape (1974). The presentation of the algorithm and the discussion of the computer implementation issues follow Dial et al. (1977), who tested a number of the most widely used minimum-path algorithms and concluded that the Moore–Pape algorithm is the most efficient for transportation networks. This reference includes a discussion and detailed presentations of many of these algorithms.

5.6 PROBLEMS

- 5.1. Show how the convex combinations algorithm can be used to solve the system optimization assignment problem discussed in Section 3.4. Describe the algorithm in detail. Should the objective function values be used as the basis for a convergence criterion?
- 5.2. Find the user-equilibrium flow pattern over the network shown in Figure P5.1. The O–D flows are:

$$1 \rightarrow 2 \quad 2 \text{ flow units}$$

$$3 \rightarrow 2 \quad 2 \text{ flow units}$$

and the volume–delay curves are $t_a = 1 + 0.15(x/a)^4$, where a is the link's number shown in the figure. Perform three iterations of the convex combinations method.

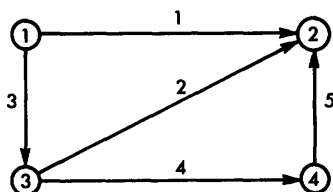


Figure P5.1

- 5.3. Streets and roadways cannot carry more than a certain amount of vehicular flow known as “capacity.”
- Formulate the UE problem assuming that each network link is associated with a capacity c_a .
 - Using the ideas outlined in the description of the feasible direction methods in Section 4.2, devise a modification to the convex combinations method described in Section 5.2 so that the algorithm would solve the capacity-constrained UE problem. You can assume that an initial *feasible* solution is given and that the performance curves are asymptotic to the capacity flow, for example, $t_a(x_a) = 1/(c_a - x_a)$.†
 - Devise a method for finding an initial feasible solution for your algorithm.

†Mathematically, it is required that the performance curves would satisfy

$$\lim_{x_a \rightarrow c_a} \int_0^{x_a} t_a(\omega) d\omega = \infty$$

as shown by Daganzo (1977c).

- 5.4. Program [5.4] defines the linearization step of the convex combinations algorithm in terms of path flows. Outline all the other algorithmic steps associated with the application of this algorithm to the UE equivalent program, in terms of path flows. Can the algorithm be executed in terms of path flows?
- 5.5. In transportation planning applications the analyst may delete and add links to the network. Discuss the problems that the forward star representation poses for easy addition and deletion of links. How can these problems be overcome?
- 5.6. Use the label-correcting algorithm to find the minimum path from node 1 to all other nodes in the network depicted in Figure 5.4.
- 5.7. Explain why the particular strategy described in the text for managing the sequence list is advantageous for transportation networks. (As an aside, note that it is not particularly advantageous for other types of networks, such as communication networks.)
- 5.8. Write a subroutine that would take as input a network in forward star representation (transferred in a COMMON block or through GLOBAL variables) and an origin node (transferred in the calling statement), and would compute the minimum paths from the origin to all other nodes. The result is to be returned in another COMMON block or a set of GLOBAL variables (of predecessors).
- 5.9. Write an input subroutine that would read network link information (including starting node, end node, and performance function parameters) in random order and store it in a forward star representation. Include a flowchart.
- 5.10. Use the results of Problems 5.8, 5.9, and 4.10 (or 4.11) to write a computer code that will find the equilibrium flows over a transportation network.
- 5.11. The operation research literature includes algorithms for finding the minimum path from every node of a network to every other node. Indicate under which circumstances such algorithms will be useful and explain why they are not used in transportation planning applications.